

From Recursive Reversal to Additive Displacement

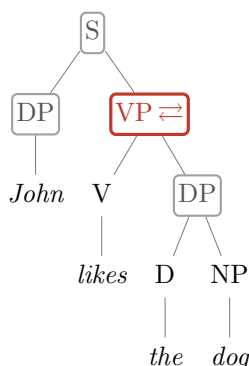
How “head-initial $\rightarrow$ head-final” becomes a permutation you compute by *counting*

Handout for the RASP-L construction (experiments/22_rasp_l_hi_hf)

Head-final (HF) reorders a head-initial (HI) sentence by reversing the two children of certain *flip* nodes, recursively. That sounds like it needs a full tree walk. This handout shows the shortcut: each token’s final position is its start position *plus a sum of independent shifts*, one per flip node it sits inside. Every example below is machine-verified against the grammar’s own generator.

1. Setup: HF is a permutation, driven by *flip* nodes

Every HI/HF rule keeps the *same words*, only reordered, so HF is a permutation of HI. A node either **flips** its two children (head $\leftrightarrow$ complement swap order) or is **invariant** (children keep order). The $\rightleftharpoons$ marks flip nodes.



HI: *John likes the dog*

Flip (reverse HI $\rightarrow$ HF):

CP_sent=[C | S], CP_rel=[C | S_gap],

VP=[V(+Adv) | obj/clause],

NP_singular=[noun | rel. clause].

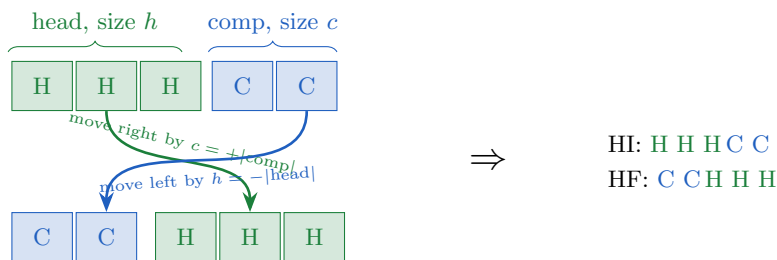
Invariant (keep order):

S=[subj|VP], DP=[D|N],

NP_singular_adj=[Adj|N], S_obj_gap=[DP|V].

2. One flip = a block swap

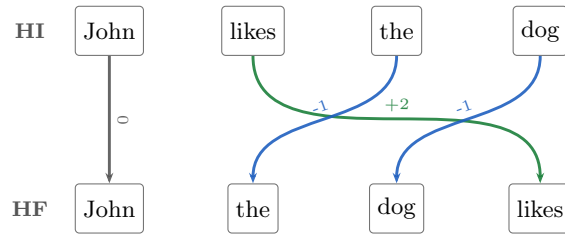
A flip node lays its children out as two adjacent blocks and swaps them:



The whole rule (one flip)

$$\text{disp}(x) = \begin{cases} +|\text{comp}| & \text{if } x \text{ is in the head child (jumps right over the complement)} \\ -|\text{head}| & \text{if } x \text{ is in the complement child (jumps left over the head)} \end{cases}$$

Worked, *John likes the dog* (VP = [*likes* | *the dog*], so $|\text{head}| = 1$, $|\text{comp}| = 2$):



A second way to read the numbers – “who crosses over me”?

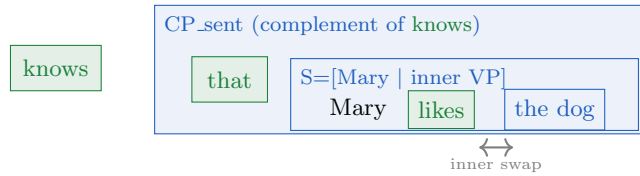
$\text{disp}(x) = (\# \text{ tokens on } x\text{'s right in HI that land left of it in HF}) - (\# \text{ that go the other way})$. For a **head** token, its *entire* complement sits to its right and all of it crosses left $\Rightarrow +|\text{comp}|$. For a **complement** token, the *entire* head sits to its left and crosses right $\Rightarrow -|\text{head}|$. This “count the crossers” view is exactly what the transformer computes with an attention head.

3. Nesting is *additive* (the key idea)

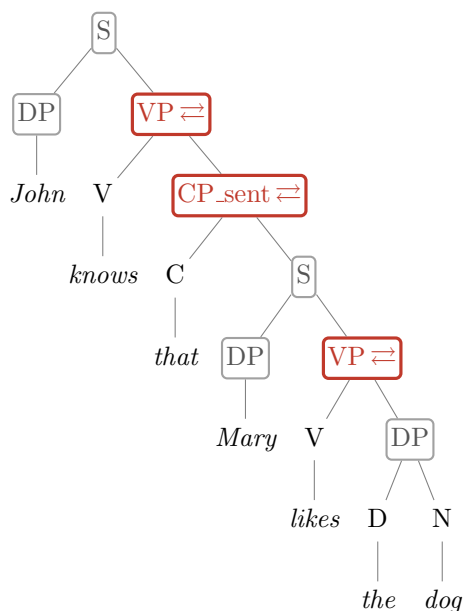
When flips nest, a token’s displacement is just the **sum** of one contribution per enclosing flip – no interaction terms. Why:

Why it adds

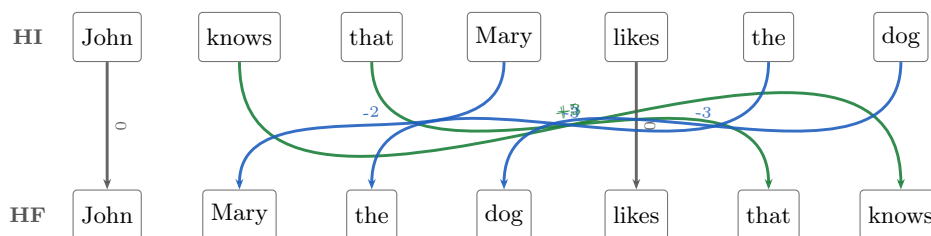
An **inner** flip only rearranges tokens *within its own span*. An **outer** flip then picks up that entire span and slides it *rigidly* – every token inside moves by the *same* amount. So each nesting level contributes an independent offset, and they stack: $\text{disp}(x) = \sum_{F \text{ a flip node enclosing } x} (+|\text{comp}(F)| \text{ if } x \in \text{head}(F), -|\text{head}(F)| \text{ if } x \in \text{comp}(F))$.



Worked, *John knows that Mary likes the dog* – three nested flips $\text{VP} \supset \text{CP_sent} \supset \text{VP}$:



tok	contributions	=disp	hf
John	(none)	0	0
knows	+5 (VP.head)	+5	6
that	-1 (VP.c) +4 (CP.head)	+3	5
Mary	-1 (VP.c) -1 (CP.c)	-2	1
likes	-1 (VP.c) -1 (CP.c) +2 (VP.head)	0	4
the	-1 -1 -1	-3	2
dog	-1 -1 -1	-3	3



Read the *likes* row: it is the head of the *inner* VP (+2), but that whole inner clause is also the complement of CP_sent (-1) and of the matrix VP (-1). The three shifts sum to 0. The embedded subject *Mary* sits under the invariant S, so it is never a flip-head/comp itself – it only collects the two -1 pushes from the flips it is *nested in*. **Subjects only ever get pushed left, never swapped.**

4. The recipe: computing an offset by hand

Everything above collapses into a four-step crank you can run on any sentence, without ever simulating the recursive reversal:

Compute $\text{disp}(x)$ in four steps

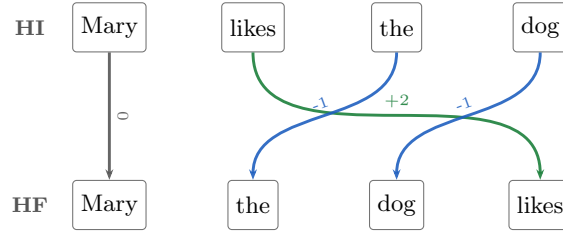
1. **Tag the tree.** Mark each internal node a **flip** (VP with an overt complement, CP_{sent}, CP_{rel}, or NP_{singular} carrying a relative clause) or **invariant** (S, DP, S_{subj_gap}, S_{obj_gap}).
2. **Size each flip.** Record $|\text{head}|$ and $|\text{comp}|$, in tokens.
3. **Collect contributions.** For a token x , walk *every* flip F whose span *contains* x (not only its parent) and add

$$+|\text{comp}(F)| \text{ if } x \in \text{head}(F), \quad -|\text{head}(F)| \text{ if } x \in \text{comp}(F).$$

4. **Sum and shift.** $\text{disp}(x) = \sum_F(\dots)$, then $\text{hf_pos}(x) = \text{hi_pos}(x) + \text{disp}(x)$; the output is the gather $\text{out}[j] = \text{hi}[x : \text{hf_pos}(x) = j]$.

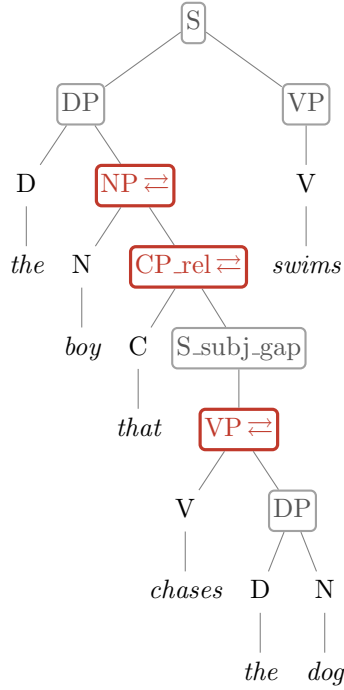
The single place this goes wrong by hand is step 3’s *contains* vs. *is a child of*: a token pays into *every* flip it sits inside, because an outer flip picks up the inner span and slides it *rigidly* (§3) – so an inner token inherits the outer $\pm$ unchanged. Two consequences worth memorizing: a **subject** sits under an invariant **S**, so it is never any flip’s head or complement – it only ever *collects* the $-|\text{head}|$ pushes of the flips it is nested in (always leftward, never a swap); and a token can be a flip’s **head** yet still not move, when the outer $-$ pushes exactly cancel its inner $+$ (the *likes* row above: $+2 - 1 - 1 = 0$).

Warm-up (one flip). “Mary likes the dog”, VP = [*likes* | *the dog*]:

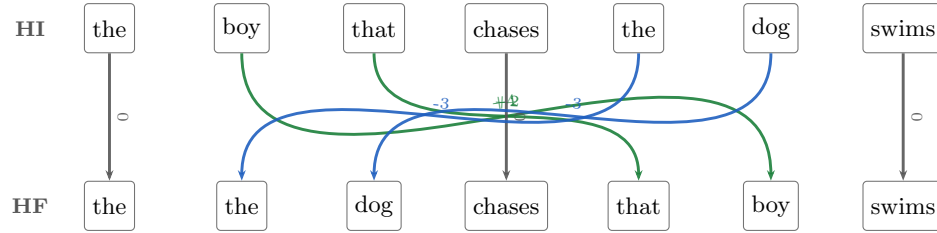


5. Same rule, a different flip: relative clauses

the boy that chases the dog swims – the outermost flip is NP_{singular} = [*boy* | *that chases the dog*], so the relative clause jumps in *front* of the noun it modifies.



tok	contributions	=disp	hf
the	(none)	0	0
boy	+4(NP.head)	+4	5
that	-1(NP.c) +3(CPrel.head)	+2	4
chases	-1 -1 +2(VP.head)	0	3
the	-1 -1 -1	-3	1
dog	-1 -1 -1	-3	2
swims	(none)	0	6



the(0) and *swims*(6) don't move: the determiner sits in an invariant DP, and the intransitive matrix verb has no complement to swap with. Same three-line rule, new node labels.

6. Why this shape is perfect for a transformer

From displacement to the program

$$\text{hf_pos}(x) = \underbrace{\text{hi_pos}(x)}_{\text{indices}} + \underbrace{\text{disp}(x)}_{\text{sum of } \pm|\text{sibling}|} \implies \text{out}[j] = \text{hi}[x : \text{hf_pos}(x) = j].$$

The displacement gives each token an *exact integer* target slot (a true permutation – no sorting), and the output is one inverse-permutation *gather*. The only thing the rule needs is, per token, the **sizes of the sibling constituents** of every flip it sits inside – i.e. the constituent *spans*. Recovering spans from the flat string is the bracket-matching in **Stage 2** of the program. Everything else – reading a token's head/complement role, and the final gather – is elementwise maps, relative-position reads, and counting: the length-generalizing core of RASP-L.

All examples verified against `grammar_oracle.py`, which computes this displacement and matches the v2 generator's hf column 100% on depth-≤ 2.